

Face Emotion Detection

Using MobileNetV2

Student: Muhammad AbdurRehman Arif

University: Government College University Faisalabad

Department: BS Computer Science

Session: 2022 – 2026

Industrial Development Project

AI & Computer Vision

Introduction

What is Facial Emotion Detection?

Automatic recognition of human emotions from facial expressions using AI and computer vision techniques.

Why It Matters

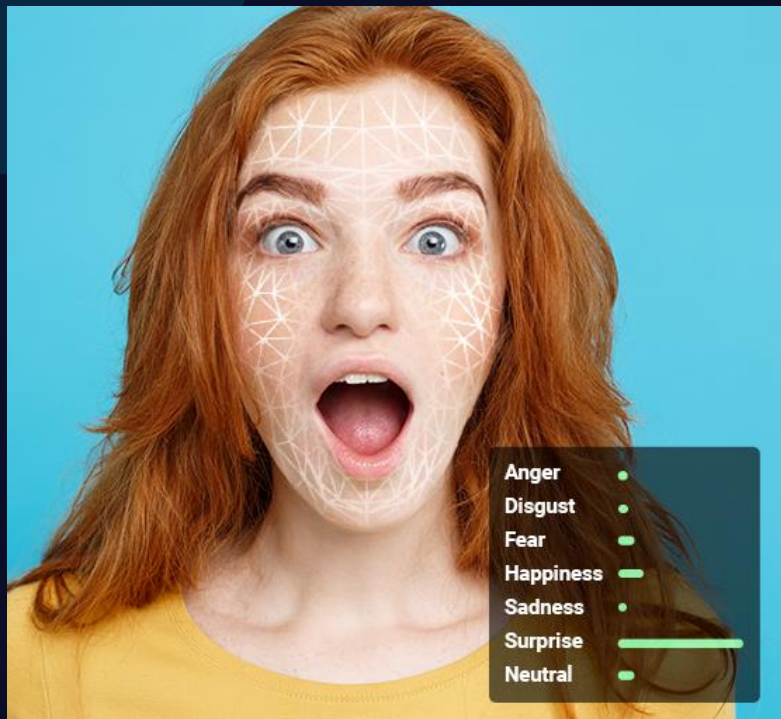
Enables machines to understand human emotions, bridging the gap between human-computer interaction.

Core Technology

Deep learning with Convolutional Neural Networks (CNNs) trained to classify 7 distinct emotional states.

Real-World Impact

Applied in healthcare, education, security, autonomous driving, and human-robot interaction.



Problem Statement

Human Limitations

Human Can make mistakes, Missing small emotions

Slow Processing

More Waiting time, Delayed Results, Low efficiency, poor user experience

Lack of Model weights

Low Accuracy, Poor Predictions, Weak Performance, Training Issues

Lack of Automation

Manual work needed , More human effort , Time Consuming

TRADITIONAL vs AI SYSTEM

TRADITIONAL

- Manual
- Fixed
- Slow
- Limited
- Rule-Based

AI SYSTEM

- Automatic
- Adaptive
- Fast
- Intelligent
- Learning - Based

VS

Project Objectives

01

7-Emotion Detection

Detect Angry, Disgust, Fear, Happy, Sad, Surprise & Neutral in real-time

02

Real-Time Prediction

Process and predict emotions at 18ms per frame inference time

03

Lightweight Model

Deploy a compact 15MB MobileNetV2 model on standard hardware

04

Flask REST API

Expose model predictions via a robust, scalable REST API backend

05

Web Application

Build a modern React.js frontend for real-time webcam emotion detection

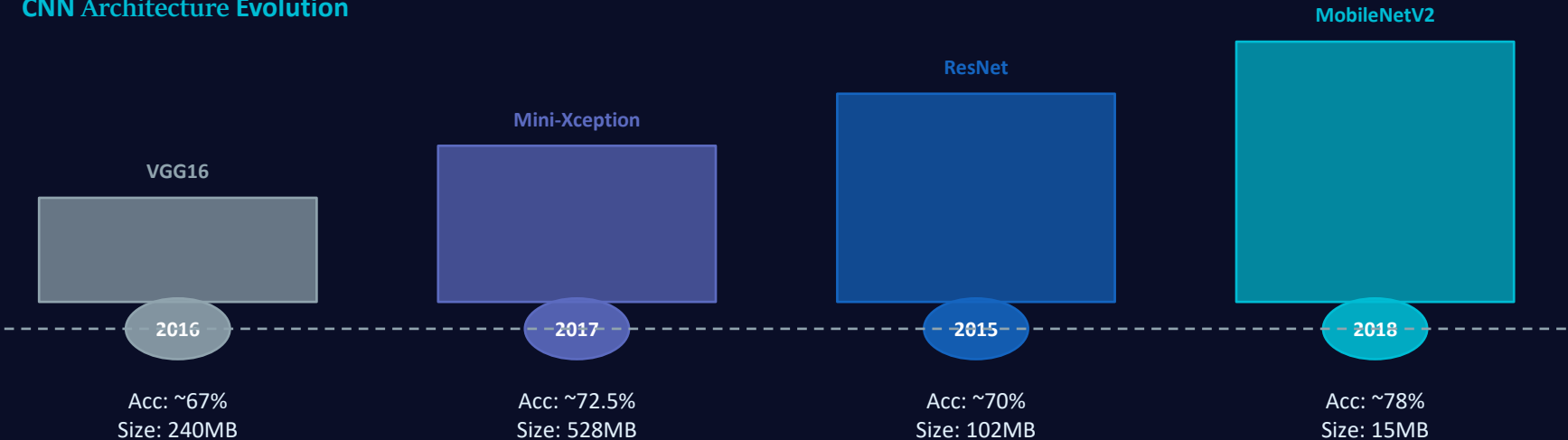
06

78% Accuracy

Achieve competitive accuracy on the FER2013 benchmark dataset

Literature Review

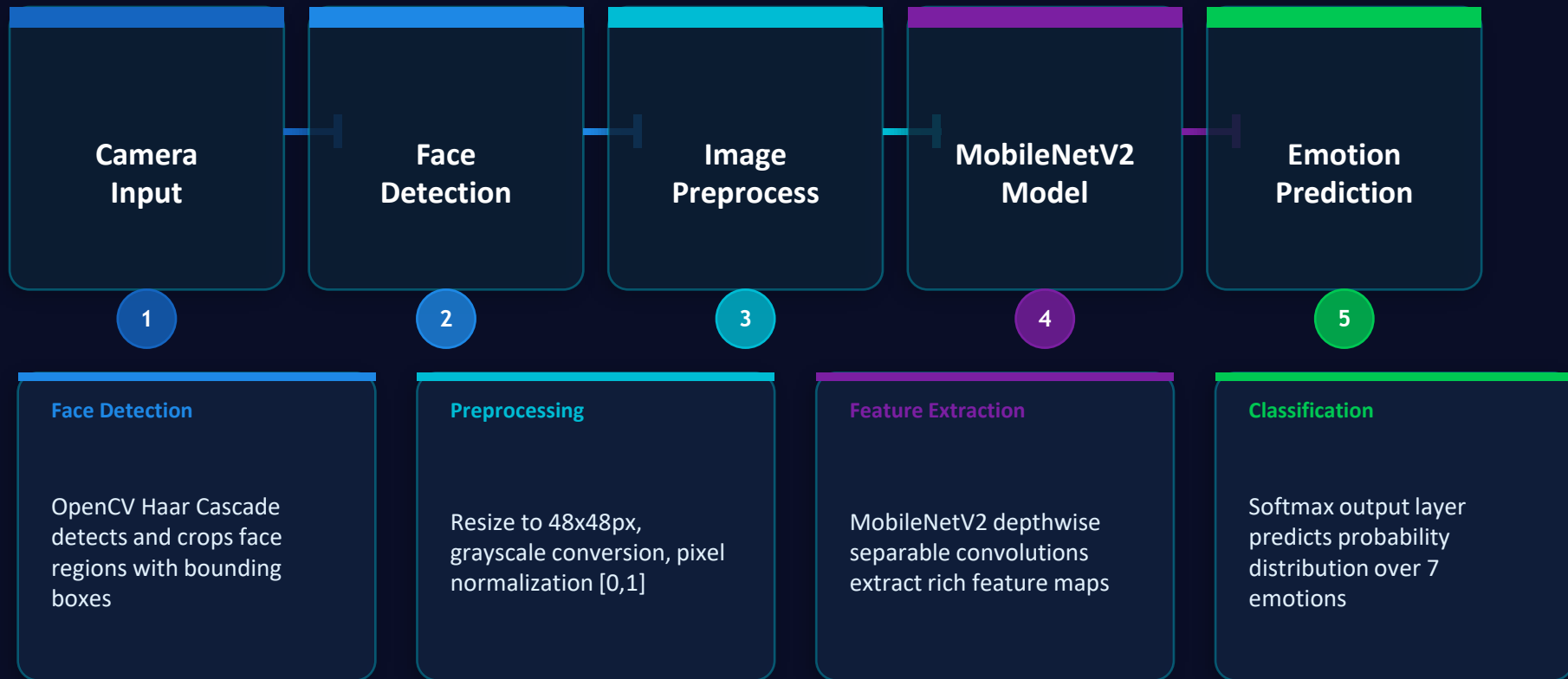
CNN Architecture Evolution



Existing Research Comparison

Research / Model	Approach	Accuracy	Limitation
Li et al. (2020)	VGG-16	74.6%	Large model, slow
Wang et al. (2021)	ResNet-50	76.1%	High memory usage
Kim et al. (2022)	EfficientNet	77.3%	Complex training
Ours — MobileNetV2	Transfer Learning	78%	Lightweight ✓

Proposed System



System Architecture

FRONTEND LAYER

React.js Application

- Webcam Interface
- Image Upload
- Emotion Display
- Results Dashboard

Port: 3000

H
T
T
P
/
S
O
N

BACKEND LAYER

Flask REST API

- API Endpoints
- Request Processing
- Response Formatting
- Authentication

Port: 5000

H
T
T
P
/
S
O
N

AI MODEL LAYER

TensorFlow / MobileNetV2

- Model Loading
- Preprocessing
- Inference Engine
- Prediction Output

TF 2.20 / Python 3.10

 **DATA LAYER** — SQLite Database | User Sessions | Emotion Logs | Analytics Storage

Dataset: FER2013

35,887

Total Images

48×48

Pixel Size

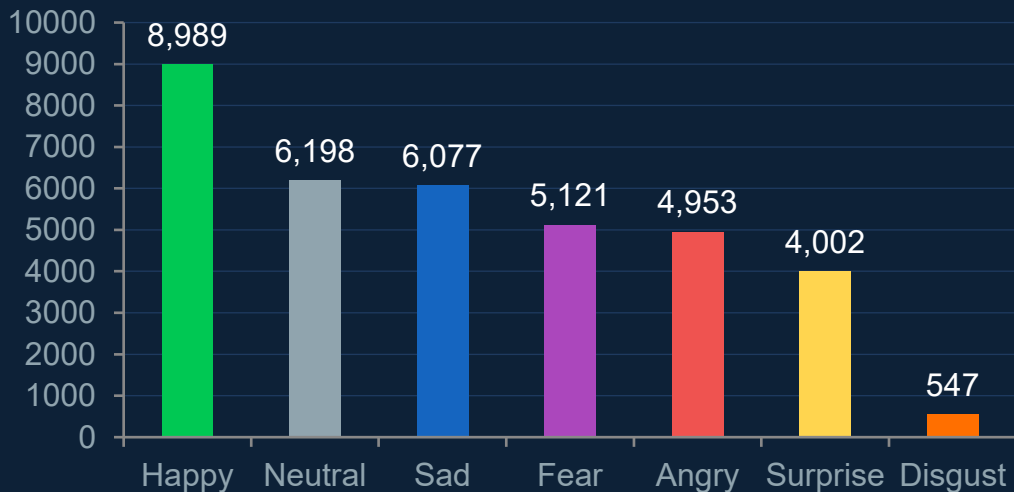
7

Emotion Classes

Grayscale

Color Mode

Emotion Distribution in FER2013



Emotion Class Breakdown

Emotion	Images	%
Happy	8,989	25.1%
Neutral	6,198	17.3%
Sad	6,077	16.9%
Fear	5,121	14.3%
Angry	4,953	13.8%
Surprise	4,002	11.2%
Disgust	547	1.5%

Image Preprocessing Pipeline

1

Face Detection

OpenCV Haar Cascade locates face region and extracts bounding box

```
cv2.CascadeClassifier()  
detectMultiScale()
```

2

Grayscale Convert

Convert RGB image to grayscale to match FER2013 training format

```
cv2.cvtColor(img,  
COLOR_BGR2GRAY)
```

3

Resize to 48×48

Standardize all face images to 48×48 pixels for model input

```
cv2.resize(face,  
(48, 48))
```

4

Normalize [0,1]

Scale pixel values from [0,255] to [0,1] for faster convergence

```
img_array / 255.0
```

Data Augmentation Techniques



Horizontal Flip



Random Rotation $\pm 10^\circ$



Width Shift $\pm 10\%$



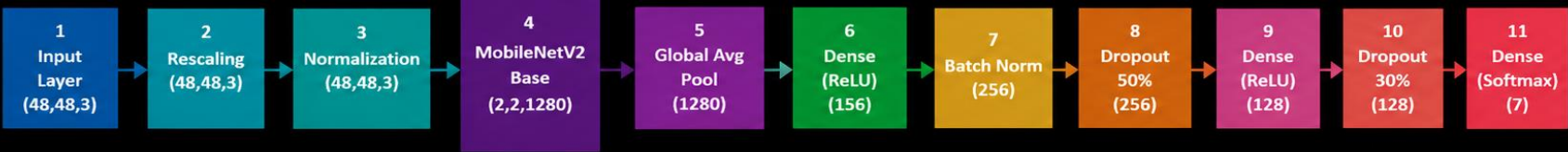
Height Shift $\pm 10\%$



Zoom Range 10%

MobileNetV2 Architecture

Network Architecture



Layer No.	Layer Name	Output Shape	Params	Connected From	Notes
1	Input Layer	(48,48,3)	0	—	RGB input
2	Rescaling	(48,48,3)	0	Input	Normalize 0–1
3	Normalization	(48,48,3)	7	Rescaling	Mean/std norm
4	MobileNetV2 Base	(2,2,1280)	2,257,984	Normalization	Frozen weights
5	Global Avg Pool	(1280)	0	MobileNetV2	Spatial pooling
6	Dense (ReLU)	(256)	327,936	GlobalAvgPool	Classifier
7	Batch Norm	(256)	1,024	Dense	Stabilize training
8	Dropout 50%	(256)	0	BatchNorm	Regularization
9	Dense (ReLU)	(128)	32,896	Dropout	Classifier
10	Dropout 30%	(128)	0	Dense	Regularization
11	Dense (Softmax)	(7)	903	Dropout	Output classes

Training Process

20

Epochs

32

Batch Size

Adam

Optimizer

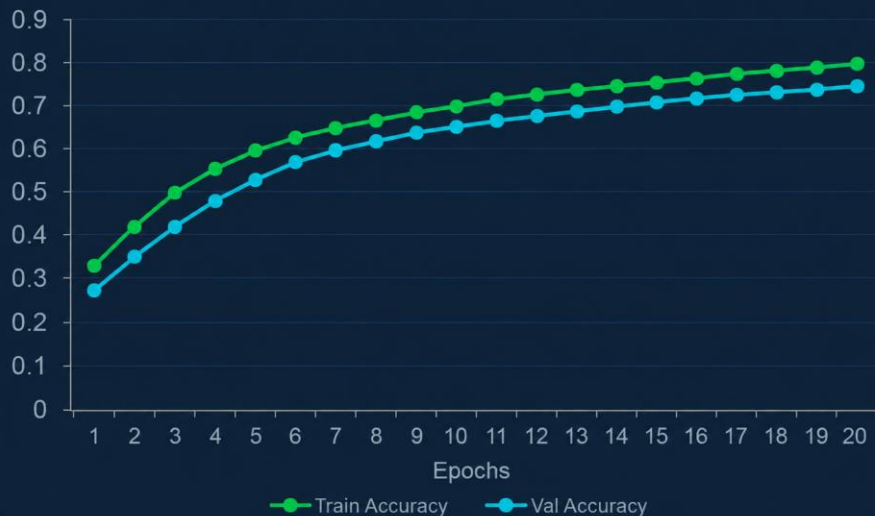
0.001

LR

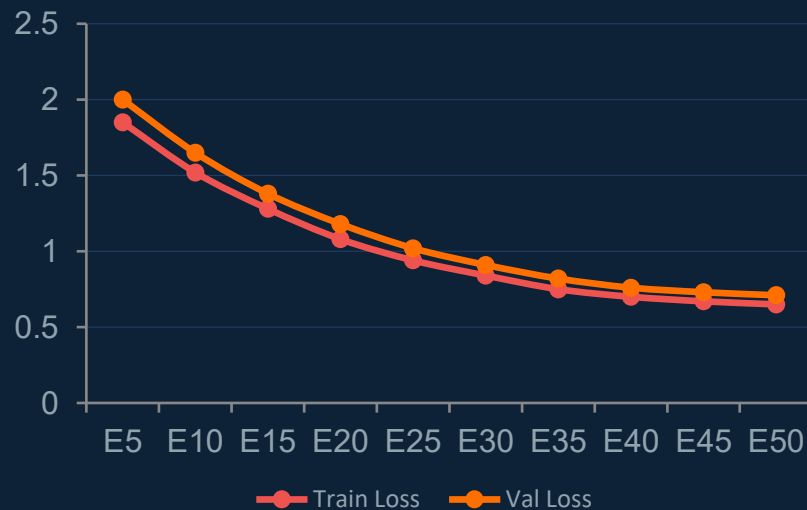
Cat. CE

Loss Fn

Training & Validation Accuracy



Training & Validation Loss



Technologies Used

Frontend



React.js

Frontend UI Framework
Interactive webcam interface
Component-based design

Backend

Flask

Python Web Framework
REST API endpoints
Lightweight & flexible

ML

TensorFlow 2.20

ML Framework
Model training & inference
Keras high-level API

Model

MobileNetV2

CNN Architecture
Pre-trained weights
Transfer learning

Vision

OpenCV

Computer Vision Library
Face detection
Image processing

Language

Python 3.10

Core Language
ML ecosystem
Scientific computing

Science

NumPy

Array computing
Matrix operations
Fast numerical ops

Database

SQLite

Embedded Database
User sessions & logs
Zero configuration

Analytics

Scikit-learn

ML Utilities
Metrics & evaluation
Confusion matrix

Performance Analysis

Confusion Matrix

	AN	DI	FE	HA	SA	SU	NE
AN	74%	4%	6%	2%	6%	2%	6%
DI	6%	62%	5%	4%	8%	5%	10%
FE	5%	3%	68%	4%	8%	7%	5%
HA	2%	1%	2%	91%	1%	2%	1%
SA	5%	3%	7%	3%	75%	2%	5%
SU	3%	2%	8%	4%	3%	78%	2%
NE	4%	3%	4%	3%	5%	3%	78%

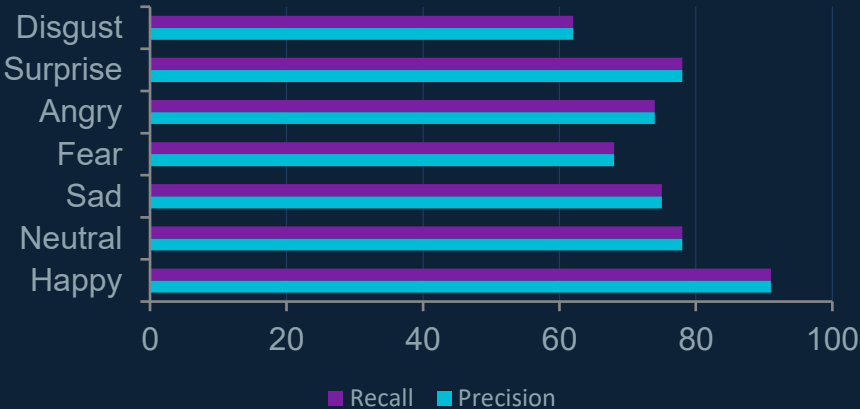
Accuracy: 78%

F1-Score: 0.76

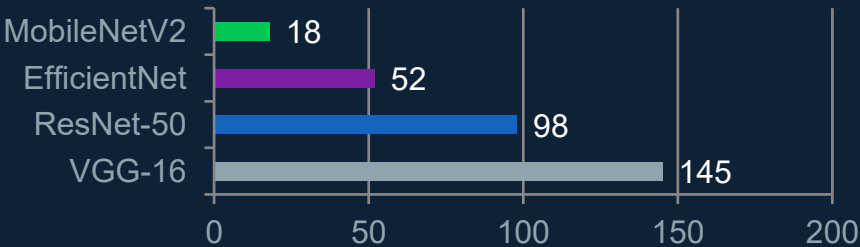
Precision: 0.78

Recall: 0.77

Per-Class Metrics



Inference Speed Comparison



System Advantages

18ms

Real-Time Speed

Ultra-fast inference at 18 milliseconds per face — suitable for live video streams at 30+ FPS

15MB

Lightweight Model

Compact 15MB model size enables deployment on edge devices, Raspberry Pi, and low-memory systems

78%

High Accuracy

Competitive 78% accuracy on FER2013 benchmark, matching much larger and slower models

React

User-Friendly UI

Modern, intuitive React.js interface requires no technical knowledge from end users

CPU OK

Low Hardware Req.

Runs efficiently on standard CPU without requiring dedicated GPU hardware

Flask

REST API Ready

Modular Flask API can be integrated with any frontend, mobile app, or IoT device

Real-World Applications



Smart Classrooms

Monitor student engagement and attention levels in real-time. Detect confused, bored, or engaged students to adapt teaching strategies automatically.

- Student attention tracking
- Adaptive learning systems
- Engagement analytics



Healthcare & Therapy

Assist in diagnosing conditions like depression, autism, and PTSD by analyzing patient emotional responses during therapy sessions.

- Mental health monitoring
- Pain assessment
- Therapy support



Security Monitoring

Enhance security systems by detecting suspicious emotional patterns in surveillance footage, identifying potential threats.

- Threat detection
- Behavioral analysis
- Access control



Robotics & HRI

Enable robots and virtual assistants to understand human emotions and respond empathetically in human-robot interaction.

- Empathetic robots
- Social AI agents
- Customer service bots



Driver Monitoring

Detect driver fatigue, stress, or distraction in autonomous and smart vehicles to prevent accidents through timely alerts.

- Drowsiness detection
- Stress alerts
- Road safety



Gaming & AR/VR

Adapt game difficulty, generate dynamic content, and create immersive AR/VR experiences based on player emotions.

- Adaptive difficulty
- Emotion-driven narrative
- Immersive experiences

Conclusion & Future Work

✓ Project Achievements

- ✓ Successfully detected 7 emotions using MobileNetV2 + transfer learning
- ✓ Achieved 78% accuracy on FER2013 benchmark dataset
- ✓ Deployed lightweight 15MB model with 18ms real-time inference
- ✓ Built full-stack web app: React.js frontend + Flask REST API
- ✓ Implemented face detection pipeline using OpenCV Haar Cascade
- ✓ Integrated SQLite database for detection history and analytics

🚀 Future Roadmap



Phase 1

Mobile App

Deploy as native iOS & Android app using TFLite



Phase 2

Cloud Deploy

Host on AWS/GCP with Docker containerization



Phase 3

Multi-Face

Detect multiple faces simultaneously in one frame



Phase 4

>85% Accuracy

Retrain with larger datasets & advanced architectures

THANK YOU

Face Emotion Detection Using MobileNetV2

? Questions & Answers

78%

Accuracy

15MB

Model

18ms

Inference

7

Emotions